

Realización de pruebas.



Caso práctico

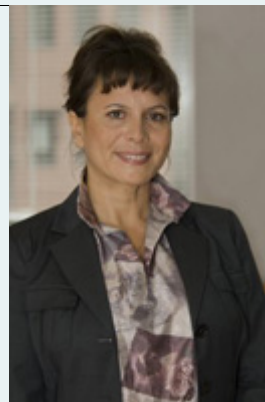
Ahora que ya sabe cómo empaquetar y distribuir aplicaciones, **Ana** cae en la cuenta de que antes de que la aplicación llegue a su destino final, a manos del cliente o clienta, tiene que probarla para asegurarse de que tenga los mínimos errores posibles.

Ada tiene muy en cuenta este aspecto. Efectivamente, desde que fundara BK Programación, la empresa siempre ha sido reconocida por la calidad de su software. Esto ha hecho contar con una amplia cartera de clientes que siempre ha confiado en los proyectos realizados por la empresa.

Las directrices de **Ada** son muy precisas: en BK Programación se hace software de calidad, y esa calidad sólo se consigue destinando recursos económicos y temporales a la realización de un severo proceso de pruebas.

Todos conocen el tipo de pruebas que hay que realizarles a las aplicaciones antes de presentarlas a los clientes. Por eso, **Ana**, que no quiere quedarse atrás, tiene que ponerse al día para participar en la estrategia de pruebas de las aplicaciones en las que está trabajando BK.

"No podemos ni pretendemos construir software 100 % libre de errores" comenta **Ada**. Pero con una estrategia adecuada de pruebas, conseguiremos minimizarlos. Nuestro software ganará calidad y los costes del proyecto disminuirán. Eso es lo fundamental.



🔍 Ministerio de

Educación y Formación

Profesional ([Elaboración propia](#))



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN
Y FORMACIÓN PROFESIONAL

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[HTML](#) [Aviso Legal](#)

1.- Pruebas de software.



Caso práctico

María va a ser la encargada de supervisar todas las pruebas a las que el software va a ser sometido durante el proceso. Les ha comentado a **Ana** y **Antonio** que pueden acompañarlos para que aprendan qué tipo de pruebas hay que realizar a la aplicación y la importancia de seguir un protocolo que las estandarice.



Ministerio de Educación y Formación Profesional (Elaboración propia)

En múltiples ocasiones, a lo largo de los distintos módulos que componen el ciclo, se comenta que la **calidad del software** que produzcamos es un requisito indispensable.

La calidad no se da a la aplicación una vez está construida, sino que es una forma de pensar y actuar del programador a lo largo de todas las fases del proyecto.

Calidad no es sólo que el software funcione bien y que vaya rápido, sino que además implica que satisfaga las expectativas de los clientes, usabilidad, confiabilidad, coste, eficiencia y cumplimiento de los estándares.

Para asegurar la calidad, debemos hacer uso de las **pruebas de software**.

Asegurar los requisitos de calidad de software lleva un coste y trabajo asociado.

La realización de pruebas de software consume tiempo y recursos, pero es fundamental para que los fallos de la aplicación sean mínimos.

Serán los usuarios clientes de nuestro proyecto quienes impongan los requisitos y los que determinen si se ha conseguido o no nuestro objetivo de proporcionar un software fiable, correcto y de calidad.

Es imposible que los desarrolladores de software podamos saber de antemano, de forma total y absoluta, las necesidades que los clientes pueden tener en cada momento y puede ocurrir que a veces malinterpretemos ciertos requisitos. Es por ello que, como fase final en la estrategia de pruebas, el software termine siendo utilizado por los clientes durante un período de prueba, para que sean ellos quienes concluyan si nuestra aplicación es la que ellos demandaron en un principio o si es necesario aplicar alguna modificación.



Ministerio de Educación y Formación Profesional (Elaboración propia)

La ausencia o la mala gestión de una estrategia de pruebas en un proyecto software pueden suponer:

- Falta de satisfacción de los clientes.
- Fallos en la explotación por defectos no detectados y eliminados a tiempo.
- Altos costes finales, por el tiempo empleado en corregir defectos.

1.1.- Objetivos.



Caso práctico



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

"No hace falta ser un experto para saber que tenemos que encontrar todos los defectos, "comenta **Antonio**". "Te equivocas", le corrige **María**. Por su propia naturaleza, el software siempre tendrá defectos. Nuestro objetivo es minimizarlos.

El objetivo de las pruebas al software es encontrar la mayor cantidad posible de errores.

Todas las aplicaciones que se construyen tienen defectos. El origen de los mismos puede ser de distinta naturaleza, pero tienen un denominador común: hay que detectarlos y eliminarlos.

En todo proyecto software se debe perseguir que los errores sean mínimos (exigir la no existencia de errores en un proyecto software es una utopía) y que el proceso de encontrarlos y corregirlos lleve asociado el mínimo coste posible.

Por ello, es imprescindible desarrollar la mejor estrategia de pruebas en nuestro proyecto.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Descubrir un error se considera el éxito de una prueba.

No sólo basta con "ejecutar código", sino que hay que hacerlo mediante requisitos, especificaciones funcionales, diseños, etc. Es decir, hay que verificar y validar el software.

El objetivo inmediato es evitar que los defectos se propaguen de unas fases a otras hasta llegar a los sistemas de producción.

En definitiva, buscamos **reducir** los riesgos por la aparición de defectos en los procesos de implantación y explotación de las aplicaciones. Todo ello mediante la detección temprana de problemas y errores. Con todo esto se pretende desarrollar un nivel de confianza sobre la disponibilidad (en tiempo) y la fiabilidad (a lo largo del tiempo) de los productos desarrollados, acorde a las expectativas del cliente.

Las pruebas deben entenderse como:

- Verificación: El producto que se está construyendo funciona correctamente; es decir, es capaz de realizar la tarea para la cual ha sido diseñado.
- Validación: El producto terminado, además de ser correcto, es conforme con lo que el cliente había esperado.



Autoevaluación

Piensa en una aplicación que se acaba de construir. Entre otras muchas cosas, tiene un buscador que conecta a una base de datos. Cuando localizamos a una persona, se muestran en una pantalla sus datos. El cliente había pedido que también se mostrara una foto de esa persona, la cual no aparece. ¿Qué podemos decir de esa aplicación?

- ☐ La aplicación ha sido verificada y validada.
- ☐ La aplicación ha sido verificada, pero no validada.
- ☐ La aplicación no ha sido ni verificada ni validada.

Incorrecto. No está validada puesto que no cumple con los requisitos del cliente.

Así es. Vas captando la idea.

Incorrecto. La aplicación funciona y hace lo que se le pide, por tanto, está verificada.

Solución

1. Incorrecto (#answer-10_63)
2. Opción correcta (#answer-10_81)
3. Incorrecto (#answer-10_84)

1.2.- Importancia.



Caso práctico

"¿Qué más dará encontrar los defectos antes o después!" dice **Antonio**. "Cuanto antes se encuentren, menos costes en corregirlos". le reprocha **María**.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

El único recurso de que disponemos los desarrolladores de software para determinar y acreditar el nivel de calidad de nuestros productos software es el proceso de pruebas.

En este proceso se ejecutan pruebas dirigidas a los componentes software individualmente y al software en su totalidad, con el objetivo de medir el grado en que el software cumple con los requisitos impuestos.

Cuanto más tarde es encontrado un defecto más cuesta (en tiempo, recursos y dinero) su corrección.

Por tanto, hemos de intentar identificar y resolver los defectos tan pronto como son originados con la mejora de los procesos abiertos.

Como resultado, entregaremos aplicaciones de alta calidad en tiempo cobertura y costes, en línea con lo establecido.

Por tanto, la estrategia de pruebas contribuye al



ahorro de coste del proyecto.

Una estrategia buena de pruebas contribuye a crear un software de mayor calidad. En concreto, contribuye a:

- Mejorar la satisfacción de la clientela.
- Incrementar la productividad.
- Disminuir los riesgos del negocio.
- Facilitar el crecimiento del negocio.

Una correcta gestión de pruebas implica una correcta gestión de la calidad.

En el caso de aplicaciones web, es a veces habitual sacar al mercado el software lo antes posible (para que no nos “pisen” nuestros competidores). Los errores pasan a un segundo plano en la primera versión y será en versiones posteriores cuando se ofrezca la aplicación mejorada. Este modelo tiene, lógicamente, el defecto de perder calidad de la marca de tu empresa y los clientes pueden desconfiar y recordarte por lo mala que fue tu primera versión.

Los defectos se introducen, en su mayoría, durante la fase de análisis de requisitos y diseño, pero suelen ser detectadas durante las pruebas de aceptación y en la explotación (que son las fases donde resulta más costosa económicamente su reparación).



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

1.3.- Estrategias de pruebas.



Caso práctico

Ada tiene definida en BK una estrategia de pruebas por la cual todo proyecto debe pasar antes de llegar a manos del cliente. Hoy, le toca a **María** explicarles a sus compañeros cuál será la estrategia a seguir para el nuevo proyecto que BK tiene asignado.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Normalmente, un proyecto software será sometido a una estrategia de pruebas. La estrategia utilizada será la que mejor se adecúe a nuestro proyecto.

Todos los desarrollos de software tienen una característica en común: lo desarrollan personas. Las personas cometemos errores y cambiamos de ideas, por eso hay que probar y volver a probar.

El coste económico relativo a un error se multiplica por un factor de 1 si se comete en la fase de análisis de requisitos, por 3-6 si es en la fase de diseño, por 10 si es en la codificación y por 40-50 si es en la fase de pruebas.

Es muy importante contar con el apoyo de la dirección del proyecto y, en su caso, por la dirección de la empresa. Deben formar parte del presupuesto, la planificación y la asignación de recursos del proyecto.

Preguntas para determinar la estrategia de pruebas a seguir: ¿Cuántos niveles de pruebas serán necesarios planificar? ¿Cuándo será conveniente parar? ¿Será mejor automatizar las tareas? Etc. Por ello, **el objetivo de la estrategia de**

pruebas es determinar los tipos de pruebas a realizar, el calendario de las mismas, los responsables, los recursos asignados y el alcance de las pruebas.

Además, será necesario conocer:

- La complejidad de la aplicación y de los módulos que la forman.
 - Plataforma en la que va a funcionar la aplicación.
 - Normativa legal pertinente.
 - Conocimientos y experiencia de los responsables de realización de las pruebas.
-

La estrategia de pruebas es un proceso complejo que consta de tres partes:

1-. Visión general de la estrategia de pruebas: según el tamaño o complejidad de la aplicación, se definen cómo:

- Las pruebas que se van a realizar.
- Las diferentes versiones de las pruebas.
- La descripción del objetivo que se pretende alcanzar en cada una de ellas

2-. Tareas en la realización de la prueba: consiste en describir:

- Las tareas propias a realizar.
- El ambiente en el que serán realizadas.
- Los datos de entrada de las pruebas y la finalización de las mismas, describiendo las acciones que deben ser ejecutadas cuando las pruebas finalicen.

3-. Resultados: consiste en documentar y revisar los resultados esperados (los que deberían de haber pasado) y los documentos reales (lo que ha pasado realmente). Si un caso de prueba falla, debe de quedar reflejado el defecto detectado.

- Según el estándar IEEE 829-1998, los documentos que se generarán durante la fase de pruebas son los siguientes:
 - **Plan de pruebas:** describe el alcance, el enfoque y el calendario de las pruebas. Qué elementos se van a probar, qué características se van a probar, quién va a realizar las pruebas y los riesgos. Especificación de pruebas: en varios documentos se recogerán los casos de prueba, las características para decir si pasa o no pasa, valores que se van a utilizar para las pruebas tanto de entrada como salida.
 - **Informes de pruebas:** consiste básicamente en un registro donde se indica la prueba que se ha realizado y los resultados que se van teniendo.
 - **Registro de pruebas:** Se registrarán los sucesos que tengan lugar durante las pruebas.
 - **Informe de incidente de pruebas:** Para cada incidente, defecto detectado, solicitud de mejora, etc., se elaborará un "informe de incidente de pruebas".
 - **Informe sumario de pruebas:** Resumirá las actividades de prueba realizadas.



Reflexiona

El coste medio de la fase de pruebas está en torno al 40-50 % del coste total del proyecto.

1.4.- Limitaciones del proceso.



Caso práctico

Juan no puede evitar pensar que el proceso de pruebas va a llevarles un tiempo precioso, tiempo que podría emplear para avanzar en otras fases del proyecto... ¿Por lo menos nos servirá para encontrar todos los errores?



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Resulta que, por su propia naturaleza, no es posible probar completamente los productos software, tanto por su complejidad como por su coste.

Por tanto, el propósito más importante es el de reducir, mediante la detección temprana de problemas, los riesgos derivados de la aparición de los posibles defectos en los procesos de implantación y explotación.

En la práctica se considera que, en software, el conjunto completo de casos de prueba es teóricamente infinito. El número de pruebas a realizar implicará un equilibrio entre recursos y tiempo.

Según los datos y tal como demuestra la experiencia, resulta que por muy cuidadosos que hayamos sido en la estrategia de pruebas siempre habrá una serie de errores que sólo aparecen cuando el cliente comienza a usarlo.



Q Ministerio de Educación y
Formación Profesional (Elaboración

Debemos tener claro que "el cliente siempre ^{propia} lleva la razón" y, por esta razón, después de un cierto tiempo en el que el cliente haya estado usando nuestra aplicación y descubra defectos, o que el programa no realice lo que se esperaba (o en la forma en que se esperaba), debemos revisar la aplicación y corregir esas taras.

Es por ello que las pruebas de aceptación, como veremos más adelante, tienen gran importancia y pueden llevarse a cabo durante incluso semanas después de haber entregado el proyecto al cliente.

Las empresas se sorprenden de que no se pueda estimar de forma realista el coste de sus proyectos. El coste asociado al proceso de pruebas no es fácilmente estimable, puesto que no hay dos proyectos iguales.

El software no puede ser 100 % libre de errores. La cuestión más difícil en la realización de pruebas es estimar el momento en que debemos parar.

Por propia definición, un proyecto de software siempre tendrá defectos y nuestra principal finalidad es minimizarlos. Para conseguirlo, hay que tener en cuenta una serie de principios:

- a.** Siempre hay falta de conocimiento del estado real de las aplicaciones durante el desarrollo de los proyectos. (Este es el motivo principal por el que nunca sabremos el coste asociado a la estrategia de pruebas).
- b.** Tener buenos conocimientos técnicos no son una garantía de éxito de un proyecto.
- c.** Hay que seguir procedimientos y actividades estandarizadas que nos guíen durante el desarrollo del proyecto y nos permitan realizar los mismos de forma repetitiva.

2.- Tipos de pruebas.



Caso práctico

Aún con sus limitaciones, las pruebas tienen ventajas incuestionables. Pero, ¿qué tipos de pruebas hay que hacerle a la aplicación?, pregunta **Ana**.

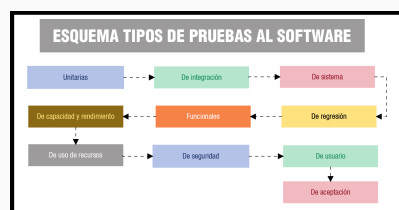
"Hay muchos tipos de pruebas y todas son muy importantes." contesta **María**. Vamos a ver cuántos tipos hay y qué función principal tiene cada uno.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Normalmente un sistema es sometido a una serie de pruebas para determinar que el software cumple la función para la cual ha sido construido y desarrollado. Según la bibliografía que consultes, encontrarás diferentes criterios a la hora de clasificar los distintos tipos de pruebas al software. La clasificación más habitual es la que comprende las siguientes de la siguiente figura.

A lo largo de la presente unidad, vas a ir viendo qué objetivos se persiguen en cada tipo de pruebas y cómo se debe planificar para que los resultados sean lo más eficientes posibles.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Los tipos de pruebas al software se pueden clasificar en:

Pruebas unitarias: se realizan a módulos o clases del programa, por separado. Tienen por objetivo probar el correcto funcionamiento de una unidad funcional, es decir, a cada unidad que compone la aplicación. Para hacerlas, debemos de tener en cuenta:

- La interfaz del módulo o componente.
- El impacto de datos globales sobre el módulo.
- Las estructuras de datos en el módulo.
- Las condiciones límite.
- Los distintos caminos de ejecución de las estructuras de control.

En el diseño de los casos de pruebas unitarias, habrá que tener en cuenta los siguientes requisitos:

- **Automatizable:** no debería requerirse una intervención manual.
- **Completas:** deben cubrir la mayor cantidad de código.
- **Repetibles o Reutilizables:** no se deben crear pruebas que solo puedan ser ejecutadas una sola vez.
- **Independientes:** la ejecución de una prueba no debe afectar a la ejecución de otra.
- **Profesionales:** las pruebas deben ser consideradas igual que el código, con la misma profesionalidad, documentación, etc.

Pruebas de integración: integran componentes y módulos del programa según un orden preestablecido. Se prueba el funcionamiento de la interrelación de todos los módulos.

■ Estrategia: Prueba de la Caja Blanca (White Box Testing)

Son fundamentales en las pruebas unitarias (probar módulos, funciones o clases individualmente) y de integración.

En este caso, se prueba la aplicación desde dentro, usando su lógica de aplicación. Se analiza y prueba directamente el código de la aplicación. Es necesario un conocimiento específico del código, para poder analizar los resultados de las pruebas. También son llamadas estructurales, para ver cómo el programa se va ejecutando, y así comprobar su correcta ejecución, se utilizan las pruebas estructurales, que se fijan en los caminos que se pueden recorrer. Pretenden comprobar que: Se van a ejecutar todas las instrucciones del programa. Que no hay código no usado. Que los caminos lógicos del programa se van a recorrer. Existen diferentes criterios:

- **Cobertura de decisiones:** se trata de crear los suficientes casos de prueba para que cada opción, resultado de una decisión, se evalúe al menos una vez a cierto y otra a falso.
- **Cobertura de condiciones:** se trata de crear los suficientes casos de prueba para que cada elemento de una condición se evalúe al menos una vez a falso y otra a verdadero.
- **Cobertura de condiciones y decisiones:** consiste en cumplir simultáneamente las dos anteriores.

- **Cobertura de caminos:** es el criterio más importante. Establece que se debe ejecutar al menos una vez cada secuencia de sentencias encadenadas, desde la sentencia inicial del programa, hasta su sentencia final. La ejecución de este conjunto de sentencias se conoce como camino. Como el número de caminos que puede tener una aplicación, puede ser muy grande, para realizar esta prueba, se reduce el número a lo que se conoce como camino prueba.

Pruebas de sistema: validan que la aplicación tenga la funcionalidad que el usuario final espera de ella. Las pruebas de sistema se ven como una "caja negra".

Pruebas de regresión: después de realizar algún tipo de modificación en el código del programa, consisten en volver a ejecutar un conjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

Pruebas funcionales (de caja negra): su objetivo es detectar errores en la implementación de los requerimientos.

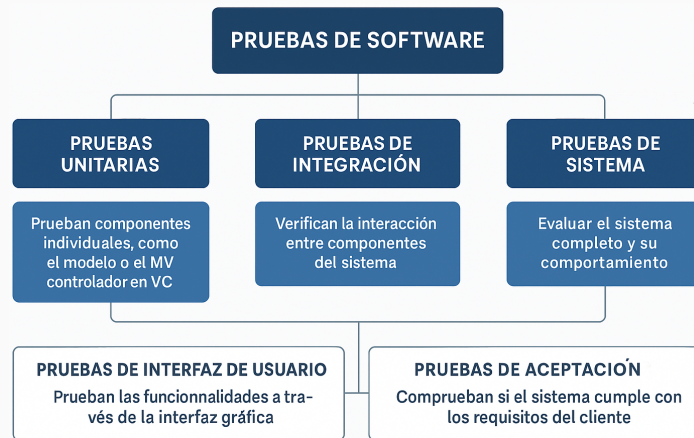
Pruebas de capacidad y rendimiento. La **prueba de capacidad** (también conocida como prueba de resistencia) ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales. La **de rendimiento** determina los tiempos de respuesta (lo rápido que realiza una tarea, un sistema en condiciones de trabajo), el espacio que ocupan los módulos en disco y en memoria, el flujo de datos que se genera, etc.

Pruebas de uso de recursos: también conocidas como *pruebas de eficiencia*, se encargan de optimizar el uso de los recursos software y hardware de una aplicación.

Pruebas de seguridad: la prueba de seguridad intenta verificar que los mecanismos de protección incorporados en nuestro sistema lo protegen de accesos no autorizados.

Pruebas de usuario: este tipo de prueba (también conocida como de usabilidad) se refiere a asegurar que la interfaz de usuario (GUI) sea amigable, intuitiva y que funcione correctamente.

Pruebas de aceptación: estas pruebas son realizadas por el cliente final. Consisten en pruebas funcionales sobre el sistema completo. Se realizan sobre el producto terminado y nunca durante el desarrollo del sistema o de la aplicación.



Q Licencia: Dominio público



Autoevaluación

Relaciona los tipos de pruebas al software con su característica más relevante, escribiendo el número asociado a la característica en el hueco correspondiente.

Ejercicio de relacionar

Tipo de prueba	Relación	Características
Prueba de Regresión.		1. Se prueba la interrelación de todos los módulos.
Prueba de Integración.		2. Las realiza el cliente sobre el producto terminado.
Prueba de Aceptación.		3. Se prueba la no existencia de errores tras una modificación.

Enviar

La prueba de integración asegura que funciona la conexión entre todos los módulos que componen la aplicación. La de regresión sirven para comprobar que un cambio no introduce errores colaterales en el sistema y la de aceptación es la prueba final, donde el cliente comprueba que la aplicación funciona y, además, de la manera en que éste había requerido desde un principio.



Ejemplo "prueba caja blanca":

Se comprueba que todas las funciones, sentencias, decisiones, y condiciones, se van a ejecutar:

- Si durante la ejecución del programa, la función es llamada, al menos una vez, el cubrimiento de la función es satisfecho.
- El cubrimiento de sentencias para esta función será satisfecho si es invocada, por ejemplo, como prueba(1,1), ya que en este caso, cada línea de la función se ejecuta, incluida $z=x$;

```
int prueba (int x,  
int y) {  
    int z=0;
```

```

if ( (x>0)      &&
      (y>0) ) {
    z=x;
}
return z;
}

```

- Si invocamos a la función con prueba(1,1) y prueba(0,1), se satisfará el cubrimiento de decisión. En el primer caso, la condición if va a ser verdadera, se va a ejecutar z=x, pero en el segundo caso, no.
- El cubrimiento de condición puede satisfacerse si probamos con prueba(1,1), prueba(1,0) y prueba(0,0). En los dos primeros casos (x<0) se evalúa a verdad, mientras que, en el tercero, se evalúa a falso. Al mismo tiempo, el primer caso hace (y>0) verdad, mientras el tercero lo hace falso.

¿En qué fases del desarrollo se hacen las pruebas?

A veces se piensa que las pruebas se hacen al final, pero en realidad:

Fase del desarrollo	Qué pruebas se realizan
Antes de programar	Definir requisitos, criterios de aceptación, casos de prueba
Durante la programación	Pruebas unitarias
Después de desarrollar un módulo	Pruebas de integración
Con la app completa funcional	Pruebas del sistema, pruebas manuales, pruebas de UI
Antes de entregar	Pruebas de aceptación (¿cumple lo que pide el usuario?)
Después del despliegue	Pruebas de regresión (que nada se rompa al cambiar algo)

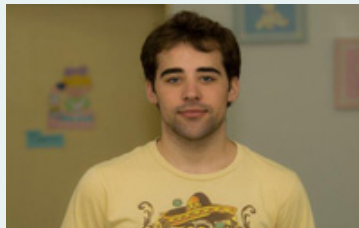
2.1- Integración.



Caso práctico

Antonio, que está estudiando el ciclo a distancia, sabe que hay que probar cada módulo del programa, de forma independiente, y comprobar que no tiene errores.

"¿Eso es todo?" Le pregunta **María**. ¿La relación entre varios módulos puede dar errores, aunque los módulos, probados de uno en uno, no los hayan dado?



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Las **pruebas de integración** son aquellas que se realizan una vez probado el funcionamiento de las partes o módulos que componen la aplicación por separado, es decir, una vez concluidas las pruebas unitarias.

Consiste en verificar que el software, en conjunto, cumple su misión y se realiza sobre la unión de todos los módulos a la vez, para probar que la interrelación entre ellos no da lugar a ningún error o defecto.

Son muy importantes por el hecho de que los módulos que componen una aplicación suelen fallar cuando trabajan de forma conjunta, aunque previamente se haya demostrado que trabajan bien de manera individual.

La integración recomendable es la llamada incremental, en la cual se van añadiendo nuevos módulos cada vez en la realización de las pruebas (y no todos de golpe). El nuevo módulo incorporado se debe probar con el conjunto de módulos que ya han sido probados.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

La prueba de integración verifica la interacción entre componentes del sistema

Dependiendo de la filosofía de integración, se definen dos técnicas: **ascendentes** y **descendentes**.

2.1.1.- Ascendentes (Bottom-Up).

Es un tipo de estrategia de pruebas de integración en el cual se comienza por los módulos de más bajo nivel hasta llegar al programa principal. Es decir, se prueba la relación entre pocos módulos y, si es correcta, se va ascendiendo de manera que cada vez habrá más módulos involucrados, llegando finalmente a probar la aplicación en su totalidad. La integración es de abajo a arriba.



Q. Ministerio de Educación y
Formación Profesional (Elaboración
propia)

El principal inconveniente es que tenemos una gran incertidumbre hasta el final de la prueba, ya que el programa en sí no existirá hasta que no se añada el último módulo. Como ventaja, no existe la necesidad de operar con resguardos.

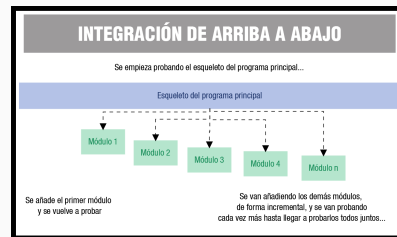
A modo de resumen, las características más importantes de la estrategia incremental ascendente son:

- Se combinan los módulos de bajo nivel en grupos con una función similar.
- Se controlan las entradas en esos módulos de bajo nivel.
- Se prueba todo el grupo.
- Se reemplazan los controladores y se combinan los grupos moviéndose hacia arriba por la estructura de módulos hasta llegar al programa principal.

Resguardo es una estructura general prevista para un módulo en particular, utilizado cuando no se conoce de antemano el código del módulo real.

2.1.2.- Descendentes (Top-Down).

Es la otra estrategia de las pruebas de integración.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Consiste en integrar los módulos moviéndose de arriba abajo por la jerarquía, comenzando desde el esqueleto del programa principal, hasta ir incorporando los módulos subordinados de forma incremental. Tras la incorporación de cada módulo, habrá que realizar la prueba de integración. El programa principal actuará como coordinador de la prueba.

La integración es, por tanto, de arriba abajo.

Ventaja: Se prueban antes los módulos más importantes. Como desventaja: es necesario trabajar con resguardos (con las complicaciones que eso conlleva).

A modo de resumen, la estrategia incremental descendente comprende los siguientes pasos:

- Se usa el módulo principal como controlador de la prueba, disponiendo de resguardos para todos los módulos subordinados.
- Se van sustituyendo los resguardos subordinados, uno a uno, por los módulos reales.
- Cada vez que se integra un módulo se realizan las pruebas.
- Tras terminar con éxito las pruebas, se reemplaza otro resguardo por otro módulo real.



Debes conocer

El orden de integración elegido (ascendente o descendente) afecta a diversos factores, como son:

- Las herramientas necesarias.
- El coste de preparación de los casos de prueba.
- El orden de codificar y probar los módulos.

- El coste de la depuración.
- La forma de preparar los casos de prueba.

La selección de una estrategia de integración depende de las características del software y del plan de proyecto. Es bastante habitual combinar las dos técnicas: usar la integración descendente para los módulos superiores de la estructura del programa y la ascendente para los niveles subordinados de bajo nivel.



Autoevaluación

Tras probar varios módulos, de forma independiente, terminamos por librarlos de errores. ¿Es necesario realizar la prueba de integración?

- ☐ No.
- ☐ Sí.

Incorrecta. El funcionamiento de cada módulo por separado no garantiza la ausencia de errores en su interrelación.

Efectivamente, hay que realizarla.

Solución

1. Incorrecto (#answer-44_63)
2. Opción correcta (#answer-44_108)

2.2.- Sistema.



Caso práctico

Llegados a este punto. ¿Podemos asegurar que una aplicación probada para un equipo y software determinado va a funcionar también en otro diferente? Es decir, ¿será la aplicación totalmente portable entre distintas plataformas hardware y software?



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Las pruebas de sistema verifican el comportamiento del sistema en su conjunto. En concreto, se comprueban los requisitos no funcionales de la aplicación:

- Seguridad.
- Velocidad.
- Exactitud.
- Fiabilidad.

También se prueban los interfaces externos con otros sistemas, utilidades, unidades físicas y el entorno operativo.

Entre las pruebas de sistema más relevantes, encontramos las que vemos a continuación.



Q INTEF (CC BY-NC-SA)

2.2.1.- Configuración.

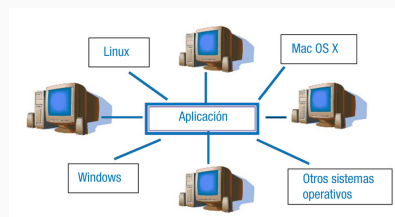
El objetivo es verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas y determinar la configuración óptima del sistema.

Estas pruebas verifican la instalación del software en el entorno de destino y comprueban el comportamiento del sistema frente a los requisitos de configuración. Además, analizan el software bajo configuraciones diferentes para diferentes usuarios.

La meta de la prueba es hacer que la aplicación falle en el cumplimiento de los requerimientos de configuración, de manera que los defectos escondidos sean identificados, analizados, arreglados y prevenidos en el futuro.

Durante estas pruebas, la persona encargada de la realización de pruebas funcionales al proyecto de software, valida que nuestro proyecto actual es capaz de soportar diferentes tipos de tecnologías de hardware, como diferentes tipos de impresoras, interfaces, topología, etc.

Estas pruebas también son llamadas pruebas de hardware o pruebas de portabilidad.



Q Ministerio de Educación y

Formación Profesional (Elaboración propia)

Actualmente se simulan las pruebas de configuración en equipos con máquinas virtuales, siendo su realización totalmente equivalente a la que se haría en varios equipos físicos.

En general, lo que se suele hacer en esta prueba es configurar una sala o laboratorio con varios equipos físicos que ejecuten diferentes combinaciones de sistemas operativos, exploradores web y otro software, para probar la aplicación con distintas combinaciones de hardware y software. Esta realización (manual) puede resultar bastante costosa, tanto en tiempo como en recursos, por lo que en la actualidad está automatizada con herramientas software que simulan todas estas situaciones sin tener que llevarlas a cabo físicamente.



Autoevaluación

El objetivo fundamental de la prueba de configuración es determinar el funcionamiento óptimo del sistema en base al uso eficiente de sus recursos.

- ☐ Sí, ese es el objetivo fundamental.
- ☐ No, el objetivo de la prueba no es ese.

No es correcto, lo que se mide es la capacidad de funcionamiento óptimo en base a distintos entornos de configuración de hardware y software.

Efectivamente, eso no es lo que se mide en esta prueba.

Solución

1. Incorrecto (#answer-50_63)
2. Opción correcta (#answer-50_115)

2.2.2.- Recuperación.

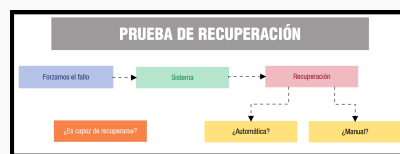
La prueba de recuperación es una prueba de sistema que fuerza el fallo del software de muchas formas y verifica que la recuperación se lleva a cabo de forma satisfactoria.

Es un parámetro muy importante de la aplicación, porque si un sistema no es capaz de recuperarse ante una entrada no válida o de un fallo súbito, la calidad de nuestro software quedará en entredicho.

La recuperación de nuestra aplicación puede llevarse a cabo de forma automática o manual. Si la recuperación es automática hay que evaluar la corrección de:

- La inicialización.
- Los mecanismos de recuperación del estado del sistema.
- Los datos.
- El proceso de arranque.

Si la recuperación requiere la intervención humana, hay que evaluar los tiempos medios de reparación (TMR) para determinar si están dentro de unos límites aceptables.



Q Ministerio de Educación y

Formación Profesional (Elaboración propia)

Un sistema tolerante a fallos evita que cese el funcionamiento de todo el sistema cuando se produce un fallo del proceso.



Autoevaluación

Las pruebas de sistema se centran en verificar los requisitos no funcionales de la aplicación. ¿Cuáles son?

- ☐ Velocidad, seguridad, exactitud y funcionalidad.

- ☐ Velocidad, seguridad, exactitud y fiabilidad.
- ☐ Validación, velocidad, seguridad y fiabilidad.

Incorrecto. La funcionalidad no es un requisito no funcional.

Así es. Vas captando la idea.

No es del todo correcta. La validación no es un requisito no funcional.

Solución

1. Incorrecto (#answer-53_63)
2. Opción correcta (#answer-53_120)
3. Incorrecto (#answer-53_123)

2.3.- Regresión.



Caso práctico

Después del éxito de la prueba de integración, **Juan** se percató de que falta un módulo en la aplicación. Lo diseñó y lo añadió.

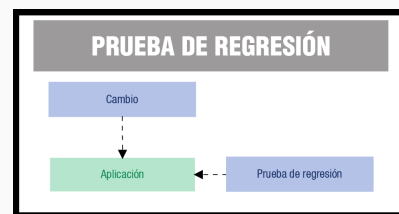
"¿Habrá que hacer otra vez todas las pruebas?" **Juan** espera que no. Sin embargo...

Resulta que cada vez que se añade un nuevo módulo a la aplicación, o cuando se modifica, el software cambia.

Como resultado de esa modificación (o adición) de módulos, podemos introducir errores en el programa que antes no teníamos. Por ello, se hace necesario volver a probar la aplicación. La prueba de regresión es volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

Así, el objetivo de las pruebas de regresión es comprobar que los cambios sobre un componente de una aplicación no introducen errores adicionales en otros componentes no modificados.

La prueba de regresión se puede hacer manualmente, volviendo a realizar un subconjunto de todos los casos de prueba o utilizando herramientas automáticas (es lo más frecuente).



El conjunto de pruebas de regresión contiene, a su vez, tres clases de prueba diferentes:

- Planificar una muestra de pruebas sobre todas las funciones del software.
- Plantear pruebas adicionales que se centren en las funciones del software que se van a ver afectadas por el cambio.
- Planificar pruebas que se centren únicamente en los componentes que han cambiado.

Es totalmente desaconsejable, por razones obvias de pérdida de tiempo, volver a realizar todas y cada una de las pruebas sobre cada uno de los componentes.

Actualmente, las pruebas de regresión son una parte integral del método de desarrollo de software conocido como Programación Extrema (Extreme Programming). Se utilizan herramientas que permiten detectar este tipo de errores de manera parcial o totalmente automatizada en cada una de las fases del desarrollo del software.

Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)



Debes conocer

4. Frameworks de Pruebas Unitarias (Base para Regresión). Estas herramientas se ejecutan directamente en Visual Studio y son la base de la regresión, permitiendo pruebas funcionales y de componentes:

- NUnit (C#/.NET): Framework de pruebas de código abierto muy popular, compatible con .NET Core y .NET Framework.
- xUnit (C#/.NET): La herramienta preferida por muchos desarrolladores modernos para .NET, más estricta que NUnit.
- MSTest (C#/.NET): El framework de pruebas nativo de Microsoft, integrado por defecto en Visual Studio.
- JUnit (Java): El estándar para pruebas en Java, integrado fácilmente en VS Code con extensiones.

En los siguientes apartados de la presente unidad profundizaremos más sobre la automatización de las pruebas, tanto de regresión como el resto.

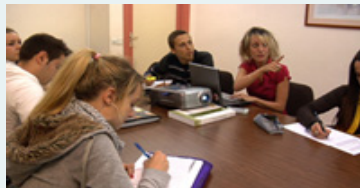
2.4.- Funcionales.



Caso práctico

"He oído hablar de las pruebas de caja negra", le comenta **Ana** a **María**. "¿A qué se refieren?"

Pruebas de caja negra son las que determinan las salidas que se esperan frente a unas entradas concretas. Son la base de las pruebas funcionales, donde sólo nos interesa que se cumplan los requerimientos, y no la forma en que se han implementado.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

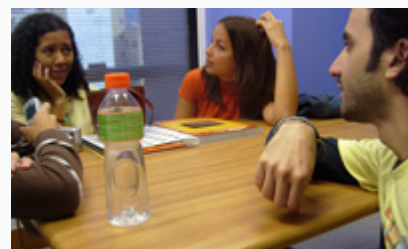
Las pruebas funcionales (o de conformidad) validan si el comportamiento observado del software es conforme con sus especificaciones (requisitos funcionales).

También son conocidas como Functional Testing y su objetivo fundamental es probar que las aplicaciones desarrolladas cumplen la función específica para la cual ha sido creada.

Normalmente, los responsables de realizar estas pruebas son los analistas de la aplicación (las personas responsables de la especificación de los requerimientos) con apoyo de los usuarios finales. Su aprobación dará paso a la fase de producción propiamente dicha.

Son **pruebas de caja negra** y las personas encargadas de realizar las pruebas, no les importa cómo se generan las respuestas, solo analizan las salidas de la aplicación frente a sus entradas.

Las pruebas funcionales intentan responder a las preguntas: "¿funciona esta utilidad de la aplicación?", "¿el usuario podrá hacer esto?".



Los sistemas que ya han sido sometidos a pruebas unitarias requieren menor tiempo para las pruebas funcionales. Esto es así porque se considera que el sistema debe ser ya bastante estable, con pocos errores críticos. Si esto no se cumpliera, deberíamos volver hacia atrás, a la etapa de pruebas unitarias. Un sistema inestable sometido a pruebas funcionales requiere altos tiempos asociados con un avance lento, además al ser pruebas de caja negra, a la persona encargada de realizar las pruebas, no le interesa el funcionamiento intermedio del sistema.



Q [Cirofono](#) (CC BY)

En este tipo de pruebas no interesa el código, ni el algoritmo, ni la eficiencia ni la construcción de elementos innecesarios. Sólo interesa verificar que las salidas que devuelve la aplicación son las esperadas, en función de las entradas que tenga.

Ejemplo

Si estamos implementando una aplicación que realiza un determinado cálculo científico, o el enfoque de las pruebas funcionales, solo nos interesa verificar que ante una determinada entrada a ese programa el resultado de la ejecución de este devuelve como resultado los datos esperados.

No se consideraría, en ningún caso, el código desarrollado ni su eficiencia, ni el algoritmo empleado, ni si hay partes de código innecesarias, etc.

Dentro de las pruebas funcionales, encontramos tres tipos:

- **Análisis de valores límite:** Como entradas, seleccionamos aquellos valores que están en el límite donde la aplicación es válida. (Si estos valores no dan problemas, el resto tampoco los dará).
- **Particiones equivalentes:** Como entradas, se seleccionará una muestra representativa de todas las posibles. El resultado será extrapolable al resto. Consiste en dividir y separar los campos de entrada según el tipo de datos y las restricciones que conllevan. De esta forma se definen unas pruebas comunes dependiendo del tipo de dato, asegurando así pruebas eficaces y completas.
 - **Ejemplo:** Campo código postal, que no es obligatorio rellenar, pero que solo admite entradas de 5 dígitos numéricos. Habrá que valorar si está o no está presente. En caso de estar presente deberíamos de confirmar que se trata de un número de 5 cifras con un rango mínimo y máximo (10000 – 99999) o también que la longitud sea 5 y solo sean números.
- **Pruebas aleatorias:** Como entradas, se selecciona una muestra de casos de prueba al azar. Se utiliza solo en aplicaciones no interactivas.



Autoevaluación

Una de las estrategias en el diseño de pruebas funcionales, de caja negra, es la de particiones variables. ¿Cuál es su principal característica?

- ☐ Selección de los valores límite de validez de las entradas.

- ☐ Escoger al azar una muestra de entradas.
- ☐ Escoger una muestra representativa de entradas.

Incorrecto. Esa estrategia es la de valores límite.

No es del todo correcto. Ese sistema es el de pruebas aleatorias.

Muy bien, así es.

Solución

1. Incorrecto (#answer-59_63)
2. Incorrecto (#answer-59_131)
3. Opción correcta (#answer-59_134)

2.5.- De capacidad y rendimiento.



Caso práctico

¿Qué pasará en los momentos en que las solicitudes a la aplicación se multipliquen? ¿Será capaz de aguantar estas cargas anormales? ¿Cuál será el tiempo de respuesta de la aplicación?

"Vamos a probarlo", asevera **María**.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

■ De capacidad:

La prueba de capacidad (también conocida como prueba de resistencia o stress) ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales. Es decir, determina hasta dónde el sistema es capaz de soportar determinadas condiciones extremas.

También tiene como finalidad determinar la capacidad del programa para soportar entradas incorrectas.

Para realizar la prueba de capacidad se recurre a los siguientes métodos: Aumentar la frecuencia de entradas para ver cómo responde la aplicación. Se ejecutan casos que requieran el máximo de memoria. Se buscan una cantidad máxima de datos que residan en disco. Se realizan pruebas de sensibilidad, intentando descubrir combinaciones de datos que puedan producir inestabilidad en el sistema.



■ De rendimiento:

Determinan los tiempos de respuesta (lo rápido que realiza una tarea un sistema en condiciones de trabajo), el espacio que ocupan los módulos en disco y en memoria, el flujo de datos que genera a través de un canal de comunicaciones, etc. La prueba de rendimiento está diseñada para probar el rendimiento del software en tiempo de ejecución dentro del contexto de un sistema integrado. Se realiza durante todos los pasos del proceso de la prueba. Incluso a nivel de unidad, se debe asegurar el rendimiento de los módulos individuales a medida que se llevan a cabo las *pruebas de caja blanca* (en donde se evalúan las salidas en función de las entradas. No se evalúa la estructura interna de la aplicación). Sin embargo, hasta que no están completamente integrados todos los elementos del sistema no se puede asegurar realmente el rendimiento del sistema. Prueban el rendimiento del software en tiempo de ejecución dentro del contexto de un sistema integrado.

- Requiere de instrumentación tanto software como hardware para los procesos de monitorización (es el proceso que asegura que nuestro proyecto va encaminado eficazmente hacia el resultado esperado) y medición.
- Se lleva a cabo durante todos los pasos de prueba.

Cuanto más se tarde en detectar un defecto de rendimiento, mayor será el coste de la solución.

Los **sistemas de tiempo real** (son aquellos al que le exigen respuestas bajo restricciones de tiempo) y los **sistemas empotrados** (diseñados para realizar determinadas funciones dedicadas, frecuentemente en un sistema de tiempo real) se tienen que ajustar a los requisitos de rendimiento.

2.6.- De uso de recursos.



Caso práctico

¿Será suficiente con cierta capacidad de memoria RAM para que un equipo soporte la aplicación? ¿Cuáles serán los demás requisitos del sistema?

"Creo que es importante, en toda aplicación, determinar los requisitos mínimos del sistema para que la aplicación funcione. Vamos a probar la eficiencia de la aplicación.



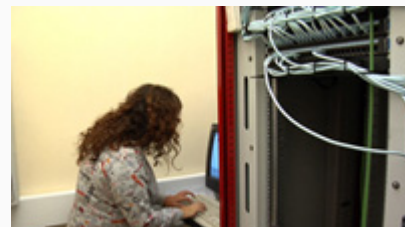
Q Ministerio de Educación y Formación Profesional (Elaboración propia)

La evolución de los sistemas informáticos lleva incorporada una mayor complejidad en su diseño y en los recursos hardware y software que requiere. Las aplicaciones requieren cada vez mayor memoria RAM y espacio en disco para funcionar correctamente y nuestros equipos habitualmente tienen limitaciones en este sentido. Es por ello importante el uso eficaz que hace una aplicación de los recursos de que dispone.

La prueba de uso de recursos es también conocida como prueba de eficiencia.

Se desarrolla un conjunto de técnicas que garantice un uso eficiente de los recursos.

Este software estará diseñado, implementado y optimizado sobre sistemas concretos. Igualmente, existe la necesidad de disponer de técnicas que aseguren el uso en sistemas que no son los originales en los que la aplicación fue diseñada. Es decir, asegurar la adaptación a otros sistemas.



Q Ministerio de Educación y Formación Profesional (Elaboración propia)

La optimización es necesaria para lograr un uso eficiente de recursos. La optimización se debe hacer tanto en el código como en el uso del código. Todo ello para lograr la adaptación del software a la arquitectura de destino y así reducir los tiempos de ejecución de forma automática.



Autoevaluación

¿Es totalmente imprescindible tener conocimiento de antemano de la arquitectura del computador destino de nuestra aplicación para hacer un

uso eficiente de recursos hardware?

- ☐ Sí.
- ☐ No.

Incorrecta. El sistema debe ser eficiente en distintas configuraciones, adaptándose a las mismas.

Efectivamente, aunque es aconsejable no es imprescindible.

Solución

1. Incorrecto (#answer-66_63)
2. Opción correcta (#answer-66_78)



Reflexiona

En la actualidad, la preocupación por la eficiencia en cuanto a recursos de la máquina (tiempo de procesador, consumo de memoria...) ha dejado paso a la preocupación por la UX (usabilidad/accesibilidad) de la aplicación (forma en la que la aplicación es presentada al cliente). Esto es así porque en estos momentos contamos con equipos con varios gigas de memoria RAM, con procesadores multinúcleo que tienen gran disponibilidad para la ejecución de los procesos (multithreading), con "discos" de incluso un TByte (SSD NVme) o más si son HDD de capacidad. Quizás la eficiencia en el uso de recursos sea algo más importante en el caso de aplicaciones web, pero de nuevo encontramos que el ancho de banda va en aumento día a día y esto va liberando a los desarrolladores de las preocupaciones sobre la economía de recursos para que se concentren en aspectos de usabilidad del software.

2.7.- De seguridad.



Caso práctico

Ana, que está "empapándose" de todo lo que acontece, cae en la cuenta de que la aplicación puede ser vulnerable frente a accesos no autorizados. Recibe la felicitación de **María**.

"Muy bien. Ha llegado el momento de probar si nuestra aplicación protegerá de aquellos que intenten acceder a ella sin las claves necesarias.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

La prueba de seguridad (también conocida como prueba de Integridad) intenta verificar que los mecanismos de protección incorporados en el sistema lo protegerán de accesos no autorizados. Mide la habilidad de un sistema para soportar ataques (tanto accidentales como intencionados) contra su seguridad. El ataque se puede ejecutar mediante los programas, los datos o los documentos de la aplicación.

Es decir, determinan los niveles de permiso de usuarios, las operaciones de acceso al sistema y acceso a los datos.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

En concreto, se intenta verificar que el sistema es robusto frente a problemas de seguridad, tales como:

- Intentar conseguir las claves de acceso de cualquier forma.
- Atacar con software a medida.
- Bloquear el sistema.
- Provocar errores del sistema, entrando durante su recuperación.

Hay que tener especial cuidado con las entradas no autorizadas cuando en el sistema de interés se maneja información de carácter privado y personal que puede atentar contra la intimidad de los seres humanos.

Durante la prueba, lo usual es que el encargado desempeña el papel de intruso y trata de violar los mecanismos de seguridad de acceso al sistema. Intenta obtener la clave de acceso de cualquier forma.

Además, también debe intentar bloquear el sistema, curiosear los datos públicos en busca de claves e introducir errores de forma consciente al sistema para ver cómo responde éste.



Autoevaluación

En la prueba de seguridad, no se considera como potencial problema:

- ☐ Bloquear el sistema.
- ☐ La tardanza en la respuesta de la aplicación.
- ☐ Intentar conseguir las claves de acceso.
- ☐ Acceder a los datos.

Incorrecta, bloquear el sistema es un aspecto que hay que revisar en esta prueba.

Efectivamente, este aspecto carece de interés en la prueba de seguridad.

No es la opción correcta. Es uno de los principales riesgos que hay que prevenir.

No es correcta, no puede darse un acceso no autorizado a los datos de la aplicación.

Solución

1. Incorrecto (#answer-72_63)

2. Opción correcta (#answer-72_84)

3. Incorrecto (#answer-72_87)

4. Incorrecto (#answer-72_90)

2.8.- De usuario.



Caso práctico

Parece que vamos terminando las pruebas de la aplicación. ¿Cómo la verá el cliente? ¿Le resultará amigable y fácil de utilizar? Siempre perseguimos la satisfacción del usuario, por lo tanto, su opinión en cuanto a la apariencia de nuestro proyecto es algo que cada vez cobra más importancia.

"Ha llegado la hora de probar la usabilidad de la aplicación", anuncia **Ada**.



Q Ministerio de Educación y Formación Profesional (Elaboración propia)

Los usuarios finales van a ser los que utilicen nuestra aplicación y, por tanto, ésta debe satisfacer sus necesidades, objetivos y expectativas.

La prueba de usuario (también conocida como de usabilidad) consiste en asegurar que la interfaz de usuario (GUI) de la aplicación sea amigable, intuitiva y que funcione correctamente. Se determina la calidad del software en la forma en la que el usuario interactúa con el sistema, considerando la facilidad de uso y satisfacción del cliente.

Se define la usabilidad de una aplicación como un parámetro que mide de qué forma los usuarios la utilizan: cómo navegan a través de sus pestañas, ventanas, buscadores, etc. Los análisis de usabilidad mostrarán lo que los usuarios realmente van a ver.

Podemos evaluar la usabilidad de una aplicación midiendo la manera en que el usuario realiza una tarea concreta, el tiempo y número de clics que supone acabarla y los errores que comete durante el proceso.

Es obvio que el estudio de la reacción de los individuos ante nuestra aplicación es un proceso complicado, pero existen varias métricas que implican un grado de fiabilidad que queda cubierto en un entorno de pruebas automatizadas.



Q Ministerio de Educación y Formación Profesional (Elaboración propia)

Para verificar la facilidad de uso de una aplicación se pueden incluir en los ensayos de pruebas:

- Encuestas y entrevistas.
- Grupos de enfoque.

- Pruebas con herramientas de software avanzado.
- Evaluaciones realizadas por expertos.

Como en otras muchas ocasiones, cuanto antes se realicen las pruebas de usabilidad, mucho mejor: no sólo resultará más barato a largo plazo, sino que se adaptará también mejor al programa (si se corrigen los problemas a tiempo no habrá que construir sobre errores).



Autoevaluación

Hemos casi terminado de probar la aplicación. Te encanta ver el resultado de tu trabajo y piensas en lo bien que ha quedado. Observas que cuando intentamos alguna opción no válida, se abre una ventana que dice “la acción no ha podido ser ejecutada con éxito”. Sea cual sea la acción no válida, el mensaje siempre es el mismo. Si nuestro cliente es una cadena hotelera y la persona que va a usar la aplicación no tiene grandes conocimientos de informática, ¿crees que el mensaje debería ser diferente?

- ☐ No, está suficientemente claro.
- ☐ Sí, debería concretar el error para que el usuario sepa dónde se ha equivocado.

No es correcto, el parámetro de usabilidad de la aplicación no satisface los requerimientos.

Efectivamente, por las características del cliente, para que no se sienta “perdido”.

Solución

1. Incorrecto (#answer-80_63)
2. Opción correcta (#answer-80_97)

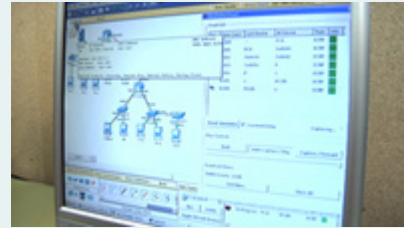
2.9.- De aceptación.



Caso práctico

Una vez todo terminado, el producto está ya listo para la entrega. En BK hay bastantes nervios. Ha sido un proyecto complejo y esperan ansiosamente la respuesta del cliente.

"Nos disponemos a validar la aplicación", anuncia **María**. "Que sea lo que el cliente requirió en su momento es el último paso para finalizar.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

La prueba de aceptación (o de validación) sirve para comprobar que el software cumple con los requerimientos iniciales y que funciona de acuerdo con lo que el cliente esperaba de él, es decir, satisfaciendo sus expectativas. Por eso, y para que la prueba sea objetiva, es el mismo cliente quien la realiza. Podemos escuchar comentarios como: "esto no era lo que yo quería" o "esto no es lo que yo necesito", e incluso "esto no es lo que yo pedí".

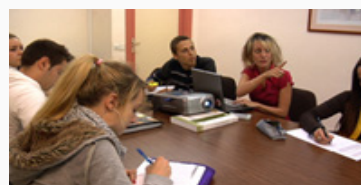
Para su ejecución se suelen emplear dos técnicas de aceptación: alfa y beta.

■ Pruebas alfa

Las lleva a cabo el cliente en el lugar donde se ha desarrollado la aplicación y en presencia del desarrollador. El cliente utilizará la aplicación como lo haría en su lugar de trabajo. Se utiliza un entorno controlado con las mismas condiciones que hay en el lugar de trabajo habitual. Una vez realizadas, se documentan los resultados.

■ Pruebas beta

Las lleva a cabo el cliente, pero ya en su lugar de trabajo. A diferencia de las pruebas alfa, el desarrollador ya no está presente durante su realización, por lo que estas pruebas se denominan "en vivo", puesto que no se ejecutan en un entorno controlado. El cliente registrará todos los problemas que encuentre durante la prueba beta e irá informando al desarrollador a intervalos regulares de tiempo. Éste realizará las correcciones y modificaciones pertinentes y preparará una nueva versión del producto.



En esta prueba se evalúa el grado de calidad del software en relación a su uso. Es definida por el usuario cliente y preparada por el equipo de desarrollo, aunque la ejecución y aprobación final corresponden al usuario.

La validación del sistema se consigue mediante la realización de pruebas de caja negra que demuestran la conformidad con los requisitos.



En la prueba de aceptación se evalúa:

- Que se satisfacen los requisitos funcionales.
- Que se alcanzan los requisitos de rendimiento.
- Que la documentación es correcta e inteligible.



Autoevaluación

**Las pruebas alfa se llevan a cabo en el lugar usual de trabajo del cliente.
¿Verdadero o falso?**

- ☐ Verdadero.
- ☐ Falso.

No es correcto, se llevan a cabo en el lugar donde se ha realizado la aplicación.

Efectivamente, las que se llevan a cabo en el lugar de trabajo del cliente son las pruebas beta.

Solución

1. Incorrecto (#answer-84_63)
2. Opción correcta (#answer-84_103)

2.10.- Manuales y automáticas.



Caso práctico

"Por favor, esto es una locura", dice Antonio. "¿Hay herramientas que automaticen todo esto? No habrá que ir haciendo prueba a prueba manualmente... ¿no?" "Claro que sí", responde Juan. "Ahora verás unas cuantas."



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

Ya sabes que la prueba consiste en la ejecución de un programa con el único objetivo de encontrar defectos. Las condiciones bajo las cuales se realizará la prueba tienen que estar prefijadas para poder evaluar los resultados que obtengamos.

Es decir, las pruebas no se hacen al azar, sino que seguirán una estrategia. En ella, será necesario definir una serie de casos de prueba, que son un conjunto de entradas, de condiciones de ejecución y resultados esperados, desarrollados para un objetivo particular.

Todos los tipos de pruebas que estamos viendo se ejecutan de manera regular con el objetivo de probar que la evolución del software desarrollado funciona perfectamente, tanto para las funcionalidades nuevas como para aquellas que ya existían. Una de las claves es que las pruebas sean automatizadas, porque si se ejecutaran manualmente, el coste de hacerlo "continuamente" sería inasumible.

¿Por qué automatizar?

La razón obvia es ganar tiempo y facilitarnos la vida.

Aparte de esta, que es evidente, los otros motivos son:

- Mejorar la calidad del producto.



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

- Detectar errores en etapas tempranas del desarrollo del proyecto.
- Disminuir el tiempo de pruebas y, por tanto, el tiempo de salida al mercado.
- Reducir costes.
- Ejecutar pruebas de manera continua.

Es importante, en caso de utilizar una herramienta que automatice el proceso de pruebas, realizar una correcta selección de la misma.

En el mercado disponemos de una amplia gama de herramientas en este sentido. Existen infinidad de herramientas gratuitas que, según las características de nuestro proyecto (número de requisitos, grado de estabilidad, organización del equipo, etc.), podemos valorar muy positivamente.

Según Meudec, las herramientas de pruebas de software persiguen automatizar tres tareas fundamentales:

1. Tareas administrativas y generación de documentación.
2. Tareas mecánicas (ejecución).
3. Tareas de generación de casos de prueba.

Siguen desarrollándose una serie de frameworks (plataforma, entorno, marco de trabajo rápido de desarrollo de aplicaciones) (como JUnit, xUnit, MSTest, ...) para automatizar pruebas unitarias y hay una multitud de entornos de desarrollo que incorporan plug-ins (complementos de aplicaciones que añaden una funcionalidad nueva y, generalmente, muy específica) y componentes que permiten su integración.

La elección de la herramienta de automatización es muy importante y depende del tipo de pruebas a automatizar, del entorno de desarrollo utilizado y del lenguaje de programación con el que hayamos codificado la aplicación.

Herramientas software para la realización de pruebas.

El panorama de las herramientas de pruebas automatizadas está en constante evolución. Algunas tendencias clave a tener en cuenta:

■ Integración con Inteligencia Artificial (IA)

Estas herramientas aprovechan los algoritmos de aprendizaje automático para:

- Aprender de los datos de prueba: La IA puede identificar patrones y tendencias, predecir posibles problemas e incluso sugerir mejoras a los guiones de prueba existentes.
- Generación automatizada de scripts de prueba: La IA puede analizar grandes cantidades de datos de prueba para generar automáticamente casos de prueba basados en la funcionalidad de la aplicación y los patrones de comportamiento del usuario, reduciendo el esfuerzo manual necesario para la creación de scripts.

■ Pruebas de desplazamiento a la izquierda y a la derecha

- Pruebas de desplazamiento a la izquierda: Esto implica integrar las pruebas automatizadas en una etapa más temprana del ciclo de vida del desarrollo (por ejemplo, pruebas unitarias, revisiones de código), lo que permite a los desarrolladores detectar y corregir errores en una fase temprana, evitando que aparezcan en etapas posteriores del desarrollo.
- Prueba de desplazamiento a la derecha: Esto implica extender la automatización más allá de las pruebas funcionales para incorporar aspectos no funcionales como pruebas de rendimiento, pruebas de seguridad, accesibilidad y monitoreo de la experiencia del usuario a lo largo del proceso de desarrollo e incluso en entornos de producción.

En la automatización de las pruebas al software, debes tener en cuenta:

- Definir los objetivos de la automatización basándonos en los objetivos de calidad.
- Elegir las pruebas que se van a automatizar.
- Seleccionar correctamente las herramientas para la automatización.
- Se trata de una inversión cuyos beneficios se observan a medio plazo.
- Debe contar con personal especializado.

Clasificación: atendiendo al tipo de prueba que realizan

Herramientas para evaluar la usabilidad sin ver el código (caja negra) y de regresión

Las pruebas funcionales o de caja negra sí pueden evaluar aspectos de usabilidad, ya que se realizan desde la perspectiva del usuario final, sin necesidad de ver el código. Se comprueba cómo interactúa el usuario con la interfaz, la claridad de los mensajes, la navegación y la respuesta del sistema ante acciones comunes. Herramientas que permiten simular escenarios reales:

- Selenium: Automatiza navegación y flujos de usuario en aplicaciones web.
- Katalon Studio: Combina pruebas funcionales y de usabilidad con interfaz intuitiva.
- Apidog: Útil para pruebas de API con enfoque en experiencia de usuario.

Pruebas unitarias y de integración en código

- JUnit, TestNG, pytest, xUnit.NET

Pruebas de rendimiento y carga

- JMeter, Gatling, LoadRunner: miden tiempo de respuesta, escalabilidad y estabilidad bajo carga.

Herramientas para pruebas de accesibilidad

Ayudan a verificar que el software sea usable por personas con discapacidades.

- axe (por Deque Systems): integrada en navegadores y herramientas de desarrollo, detecta problemas de accesibilidad según estándares WCAG.
- Lighthouse (de Google): incluye auditorías de accesibilidad dentro de Chrome DevTools, generando informes con recomendaciones.
- Pa11y: herramienta de código abierto que permite monitoreo continuo de accesibilidad en sitios web.
- ARC Toolkit: compatible con entornos de desarrollo y pruebas

Herramientas de prueba unitaria: Alternativas en Visual Studio

En el ecosistema .NET, MSTest y xUnit son las dos opciones más extendidas, y aunque ambas tienen el mismo fin (un código sin errores), sus filosofías son distintas. El framework xUnit lleva tiempo ganando terreno a MSTest por ser la opción más moderna que ofrece un aislamiento más estricto (instancia nueva por prueba). Aunque MSTest es el framework integrado de Microsoft, más tradicional en el ecosistema VS y por ello puede servir perfectamente para nuestros objetivos. Si embargo, es la propia Microsoft la que ya está recomendando xUnit para los desarrollos basados en .NET.

Característica	MSTest (El Clásico)	xUnit (El Estándar Moderno)

Origen	Creado por Microsoft. Viene "de serie".	Creado por los desarrolladores de NUnit. Comunitario.
Atributos de Clase	Requiere [TestClass].	No necesita atributos en la clase.
Atributos de Método	[TestMethod]	[Fact] (prueba simple) o [Theory] (con datos).
Ciclo de Vida	Usa [TestInitialize] y [TestCleanup].	Usa el Constructor e IDisposable .
Paralelización	Configuración manual y más rígida.	Nativa y automática. Mucho más rápido.
Filosofía	Tradicional, similar a JUnit (Java).	Se basa en convenciones y simplicidad. Busca código limpio y funcional.

Manual MSTest (pruebas unitarias con VS)

Fuente oficial Microsoft Learn:

- [Introducción a las pruebas unitarias \(https://learn.microsoft.com/es-es/visualstudio/test/getting-started-with-unit-testing?view=visualstudio&tabs=dotnet%2Cmstest\)](https://learn.microsoft.com/es-es/visualstudio/test/getting-started-with-unit-testing?view=visualstudio&tabs=dotnet%2Cmstest).
- [Conceptos básicos de pruebas unitarias \(https://learn.microsoft.com/es-es/visualstudio/test/unit-test-basics?view=visualstudio\)](https://learn.microsoft.com/es-es/visualstudio/test/unit-test-basics?view=visualstudio).
- [UD6_RA8PruebasVisualStudio.pdf \(Ventana nueva\)](#)
(UD6_RA8PruebasVisualStudio.pdf).

Caso práctico en entorno Visual Studio.

¿Por dónde empezar las pruebas de un proyecto Windows Forms .NET desarrollado siguiendo el patrón MVC?

Aunque nuestra aplicación tenga interfaz gráfica, no se empieza probando la interfaz. Vamos a aprovechar la separación Modelo–Controlador–Vista para que:

- El Modelo y el Controlador se pueden probar automáticamente con tests unitarios.
- La Vista se probará a posteriori con pruebas de interfaz opcionales.

La estructura: src y tests

Para que la Guía de Referencia de nuestra aplicación sea lo más adecuada posible, la organización física de los archivos debe cumplir con la estructura estándar de la industria (basada en las recomendaciones de la comunidad .NET).

La mejor forma de organizar una solución es separar el código que se entrega al cliente (Source) del código que solo usamos nosotros para validar (Tests). Podemos tomar como modelo el siguiente árbol de directorios:

```
/Carpeta_Raiz_Solucion
├── NombreSolucion.sln <-- Único archivo en
la raíz
|
├── /src
├── /NombreSolucion.App
├── /Controlador
|   └── clase.cs
├── /Modelo
|   ├── Clase1.cs
|   └── Clase2.cs
├── /Vistas
|   ├── ClaseFormulario.cs (La clase parcial
de código)
|   ├── ClaseFormulario.Designer.cs
|   └── InterfazVista.cs (Interfaz de la
vista)
├── /Recursos
|   ├── /Icons
|   │   ├── fichero.ico
|   │   └── fichero.png
|   └── /Config
|       └── fichero.json|
|
└── Program.cs (Punto de entrada)
|
```

```
└─ NombreProyecto.App.csproj |
└─ /tests <-- Carpeta para validación
└─ /NombreSolucion_Proyecto.Tests <--
Subcarpeta del proyecto xUnit
└─ NombreProyecto.Tests.csproj
└─ ModeloTests.cs
```

La Pirámide de Pruebas de una aplicación MVC

Para seguir una estrategia eficiente, nos basaremos en una jerarquía de pruebas. No vamos a probar todo desde la interfaz (UI) al principio, porque las pruebas de UI son lentas y frágiles.

- Fase 1: Pruebas Unitarias automáticas del modelo de datos. Herramienta: Proyecto de MSTest o xUnit en Visual Studio.
 - Objetivo: Verificar que el modelo respeta los límites.
- Fase 2: Pruebas de Integración.
 - Las pruebas de integración son totalmente automatizables siempre que: los componentes a probar tengan interfaces claras (como en el patrón MVC), no dependan directamente de la interfaz gráfica, y la lógica clave esté separada en clases testables.
 - Cuando terminas un conjunto de módulos. Comprueba que dos o más componentes funcionan bien juntos. En un proyecto MVC: Modelo + Controlador.
Ejemplo práctico editor de textos: El controlador debe decirle al modelo que abra el archivo (Controlador + sistema de archivos) y luego notificar a la vista que actualice el texto.
 - Objetivo: Verificar que cuando el controlador recibe un evento de la vista, actualiza el modelo y luego llama a los métodos de la interfaz de la vista.
- Fase 3: Pruebas de integración con simulacros (Mocks)
- Instalar librería Moq: Herramientas > Administrador de paquetes NuGet > Consola del Administrador de paquetes:
 - Asegurarse de que en "Proyecto predeterminado" esté seleccionado el proyecto de pruebas.
 - Escribe: Install-Package Moq y presiona Enter.



Para saber más:
[Introducción a Moq | Pruebas automatizadas de software.](#)

- Fase 4: Pruebas de Sistema (manuales)
 - Objetivo: Se prueba la aplicación completa funcionando como un usuario real (rendimiento, estabilidad, funcionalidades completas, ...). Ejemplo práctico en una aplicación como el editor de textos Bloc de notas: Abrir un archivo de 5 MBytes, grande para un fichero de texto.

Implementación del Proyecto de Pruebas en el entorno VS:

1. Clic derecho en tu Solución -> Agregar -> Nuevo Proyecto.

- 2.** Busca Proyecto de pruebas de MSTest (o xUnit).
- 3.** Agrega una Referencia del proyecto de pruebas a tu proyecto VS.